

Chapter 4

Data-Level Parallelism in Vector, SIMD, and GPU Architectures

Introduction

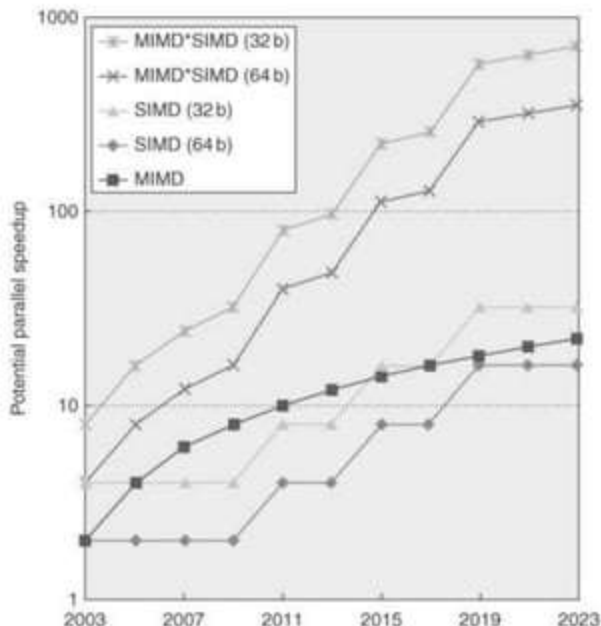
- SIMD architectures can exploit significant data-level parallelism for:
 - matrix-oriented scientific computing
 - media-oriented image and sound processors
- SIMD is more energy efficient than MIMD
 - Only needs to fetch one instruction per data operation
 - Makes SIMD attractive for personal mobile devices
- SIMD allows programmer to continue to think sequentially

SIMD Variations

- Vector architectures
- SIMD extensions
 - MMX: Multimedia Extensions (1996)
 - SSE: Streaming SIMD Extensions
 - AVX: Advanced Vector Extension (2010)
- Graphics Processor Units (GPUs)

SIMD vs MIMD

- For x86 processors:
 - Expect two additional cores per chip per year
 - SIMD width to double every four years
 - Potential speedup from SIMD to be twice that from MIMD!

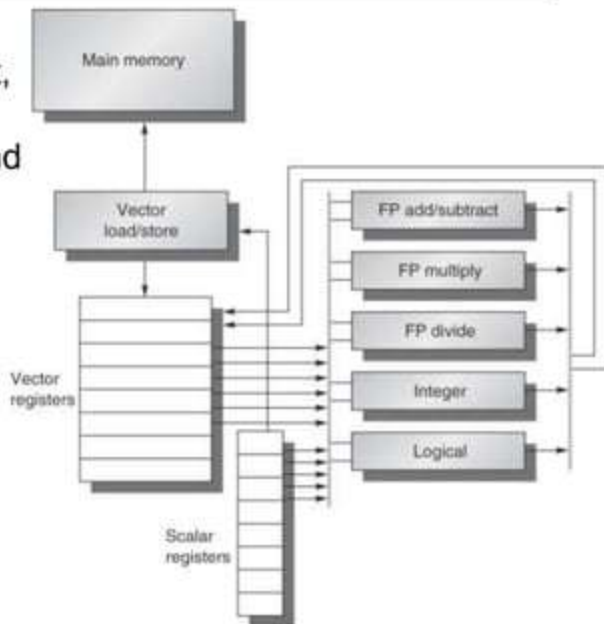


Vector Architectures

- Basic idea:
 - Read sets of data elements into “vector registers”
 - Operate on those registers
 - Disperse the results back into memory
- Registers are controlled by compiler
 - Register files act as compiler controlled buffers
 - Used to hide memory latency
 - Leverage memory bandwidth
- Vector loads/stores deeply pipelined
 - pay for memory latency once per vector ld/st!
- Regular loads/stores
 - pay for memory latency for each vector element⁵

Example: VMIPS

- Vector registers
 - Each register holds a 64-element, 64 bits/element vector
 - Register file has 16 read ports and 8 write ports
- Vector functional units
 - Fully pipelined
 - Data and control hazards are detected
- Vector load-store unit
 - Fully pipelined
 - Words move between registers
 - One word per clock cycle after initial latency
- Scalar registers
 - 32 general-purpose registers
 - 32 floating-point registers



VMIPS Instructions

# C code	# Scalar Code	# Vector Code
for (i=0; i<64; i++) C[i] = A[i] + B[i];	LI R4, 64 loop: L.D F0, 0(R1) L.D F2, 0(R2) ADD.D F4, F2, F0 S.D F4, 0(R3) DADDIU R1, 8 DADDIU R2, 8 DADDIU R3, 8 DSUBIU R4, 1 BNEZ R4, loop	LI VLR, 64 LV V1, R1 LV V2, R2 ADDV.D V3, V1, V2 SV V3, R3

VMIPS Instructions

- Example: DAXPY

L.D	F0,a	;load scalar a
LV	V1,Rx	;load vector X
MULVS.D	V2,V1,F0	;vector-scalar mult
LV	V3,Ry	;load vector Y
ADDVV	V4,V2,V3	;add
SV	Ry,V4	;store result

- In MIPS Code

- ADD waits for MUL, SD waits for ADD

- In VMIPS

- Stall once for the first vector element, subsequent elements will flow smoothly down the pipeline.
- Pipeline stall required once per vector instruction!

VMIPS Instructions

- Operate on many elements concurrently
 - Allows use of slow but wide execution units
 - High performance, lower power
- Independence of elements within a vector instruction
 - Allows scaling of functional units without costly dependence checks
- Flexible
 - 64 64-bit / 128 32-bit / 256 16-bit, 512 8-bit
 - Matches the need of multimedia (8bit), scientific applications that require high precision.

Vector Execution Time

- Execution time depends on three factors:
 - Length of operand vectors
 - Structural hazards
 - Data dependencies
- VMIPS functional units consume one element per clock cycle
 - Execution time is approximately the vector length

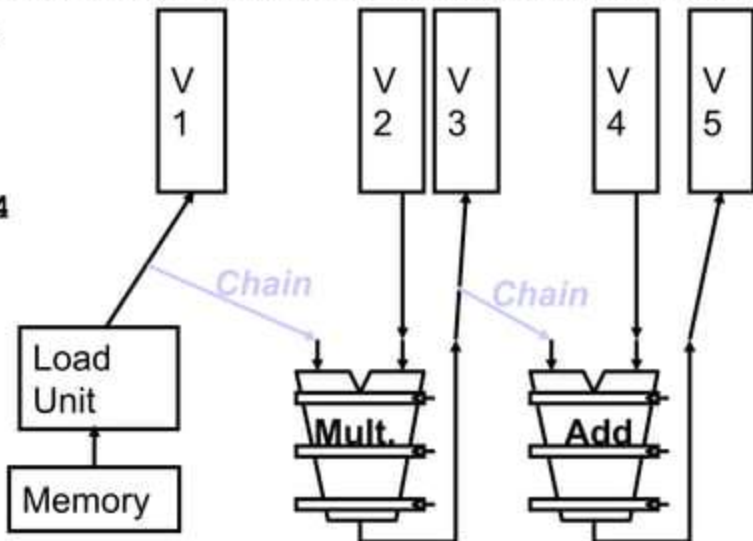
Convoy

- Set of vector instructions that could potentially execute together
- Must not contain structural hazards
- Sequences with read-after-write dependency hazards should be in different convoys
 - however can be in the same convoy via chaining

Vector Chaining

- Vector version of register bypassing
- Chaining
 - Allows a vector operation to start as soon as the individual elements of its vector source operand become available
 - Results from the first functional unit are forwarded to the second unit

LV v1 ↘
MULV v3, v1, v2
ADDV v5, v3, v4



Vector Chaining Advantage

- Without chaining, must wait for last element of result to be written before starting dependent instruction



- With chaining, can start dependent instruction as soon as first result appears



Convoy and Chimes

- Chime
 - Unit of time to execute one convoy
 - m convoys executes in m chimes
 - For vector length of n , requires $m \times n$ clock cycles

Example

LV	V1,Rx	;load vector X
MULVS.D	V2,V1,F0	;vector-scalar mult
LV	V3,Ry	;load vector Y
ADDVV.D	V4,V2,V3	;add two vectors
SV	Ry,V4	;store the sum

- Convoys:

1	LV	MULVS.D
2	LV	ADDVV.D
3	SV	

- 3 chimes, 2 FP ops per result, cycles per FLOP = 1.5
- For 64 element vectors, requires $64 \times 3 = 192$ clock cycles

Challenges

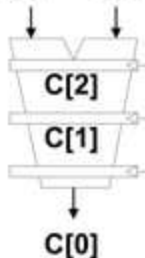
- Start up time
 - Latency of vector functional unit
 - Assume the same as Cray-1
 - Floating-point add => 6 clock cycles
 - Floating-point multiply => 7 clock cycles
 - Floating-point divide => 20 clock cycles
 - Vector load => 12 clock cycles

Vector Instruction Execution

ADDV C, A, B

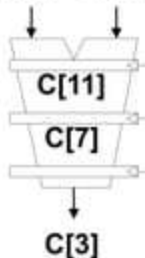
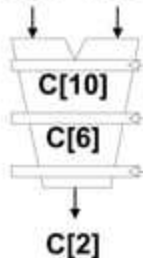
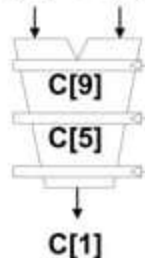
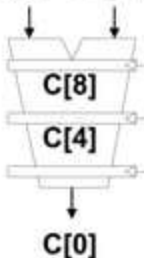
*Execution using
one pipelined
functional unit*

A[6] B[6]
A[5] B[5]
A[4] B[4]
A[3] B[3]



*Four-lane
execution using
four pipelined
functional units*

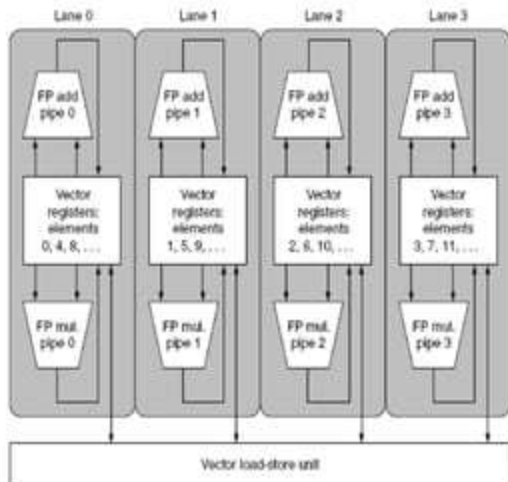
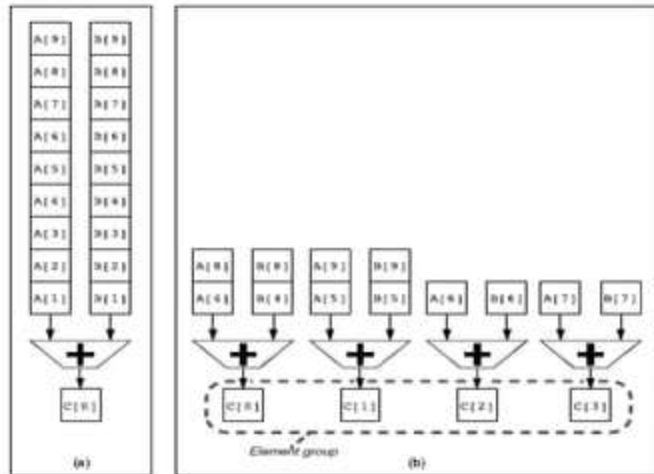
A[24] B[24] A[25] B[25] A[26] B[26] A[27] B[27]
A[20] B[20] A[21] B[21] A[22] B[22] A[23] B[23]
A[16] B[16] A[17] B[17] A[18] B[18] A[19] B[19]
A[12] B[12] A[13] B[13] A[14] B[14] A[15] B[15]



Multiple Lanes

• Element n of vector register A is “hardwired” to element n of vector register B

- Allows for multiple hardware lanes
- No communication between lanes
- Little increase in control overhead
- No need to change machine code



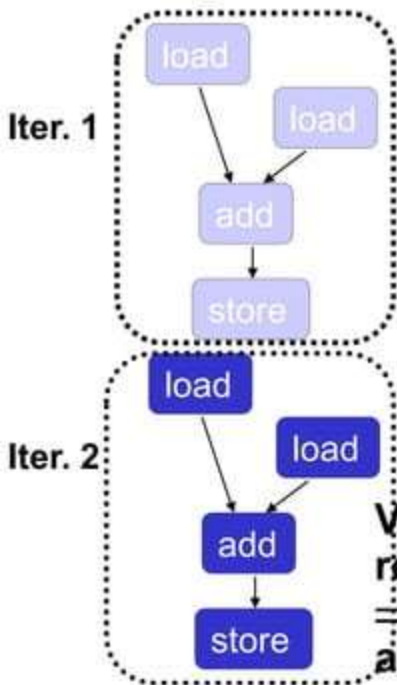
Adding more lanes allows designers to tradeoff clock rate and energy without sacrificing performance!

Automatic Code Vectorization

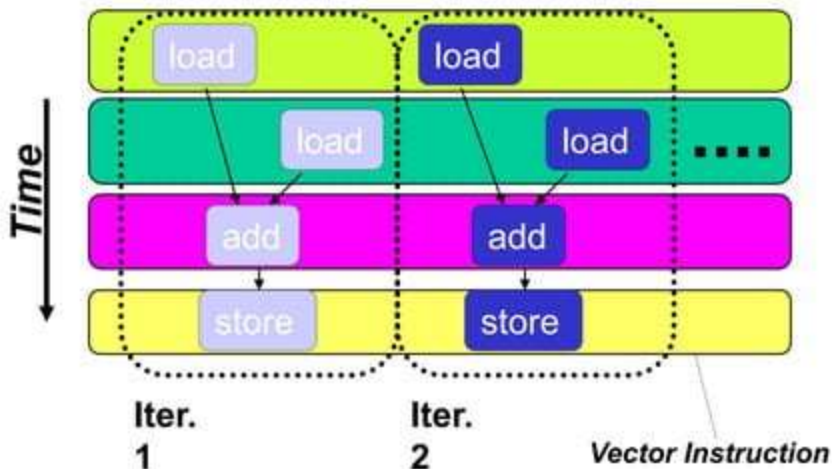
```
for (i=0; i < N; i++)
```

```
  C[i] = A[i] + B[i];
```

Scalar Sequential Code



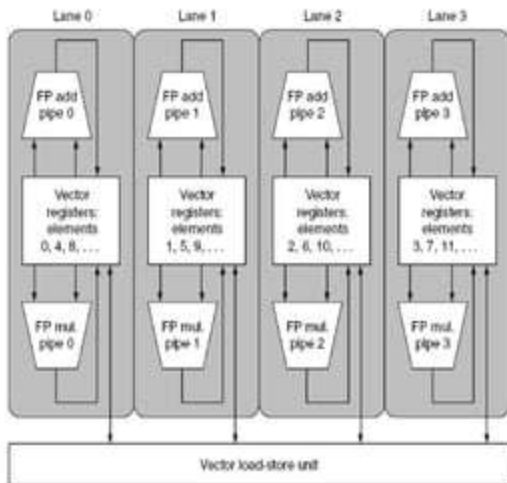
Vectorized Code



Vectorization is a massive compile-time reordering of operation sequencing
⇒ requires extensive loop dependence analysis

Multiple Lanes

- For effective utilization
 - Application and architecture must support long vectors
 - Otherwise, they will execute quickly and run out of instructions requiring ILP



Vector Length Register

- Vector length not known at compile time?
- Use Vector Length Register (VLR)
- Use strip mining for vectors over maximum length:

low = 0;

VL = (n % MVL); /*find odd-size piece using modulo op % */

for (j = 0; j <= (n/MVL); j=j+1) { /*outer loop*/

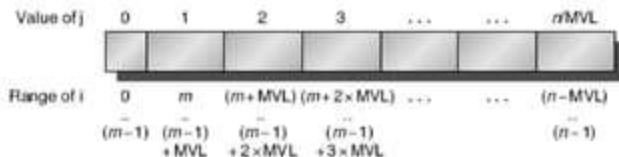
 for (i = low; i < (low+VL); i=i+1) /*runs for length VL*/

 Y[i] = a * X[i] + Y[i]; /*main operation*/

 low = low + VL; /*start of next vector*/

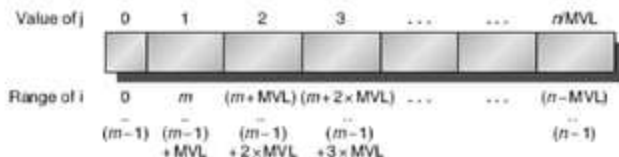
 VL = MVL; /*reset the length to maximum vector length*/

}



Maximum Vector Length

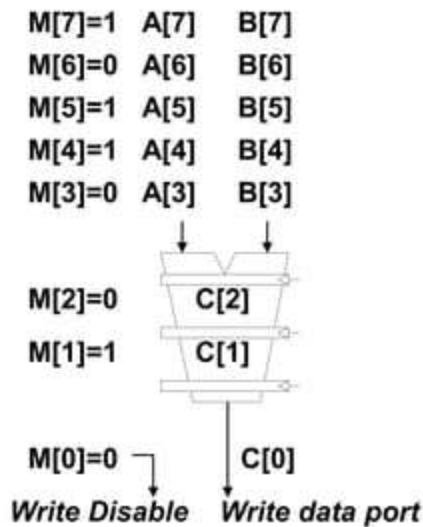
- Advantage:
 - Determines the maximum number of elements in a vector for a given architecture
 - Later generations may grow the MVL
 - No need to change the ISA



Masked Vector Instruction Implementations

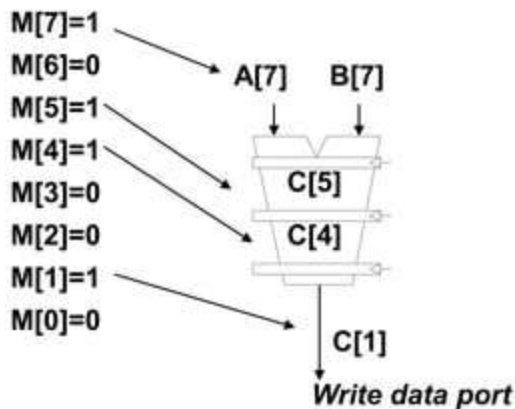
Simple Implementation

- execute all N operations, turn off result writeback according to mask



Density-Time Implementation

- scan mask vector and only execute elements with non-zero masks



Vector Mask Register

- Consider sparse matrix operations!:

for (i = 0; i < 64; i=i+1)

if (X[i] != 0)

X[i] = X[i] - Y[i];

- Use vector mask register to “disable” elements:

LV	V1,Rx	;load vector X into V1
LV	V2,Ry	;load vector Y
L.D	F0,#0	;load FP zero into F0
SNEVS.D	V1,F0	;sets VM(i) to 1 if V1(i)!=F0
SUBVV.D	V1,V1,V2	;subtract under vector mask
SV	Rx,V1	;store the result in X

- GFLOPS rate decreases!

Vector Mask Register

- VMR is part of the architectural state
- Rely on compilers to manipulate VMR explicitly
- GPUs get the same effect using HW!
 - Invisible to SW
- Both GPU and Vector architectures spend time on masking!

Memory Banks

- Memory system must be designed to support high bandwidth for vector loads and stores
- Spread accesses across multiple banks
 - Control bank addresses independently
 - Load or store non sequential words
 - Support multiple vector processors sharing the same memory
- Example:
 - 32 processors, each generating 4 loads and 2 stores/cycle
 - Processor cycle time is 2.167 ns, SRAM cycle time is 15 ns
 - How many memory banks needed?

Memory Banks

- 6 mem refs / processor
- $6 \times 32 = 192$ mem refs
- $15 / 2.167 = 6.92$ processor cycles pass for one SRAM cycle
- Therefore around $7 \times 192 = 1344$ banks are needed!
 - Cray T932 has 1024 banks
 - It couldn't sustain full bandwidth to all processors
 - Replaced SRAM with pipelined asynchronous SRAM (halved the memory cycle time)

Stride: Multidimensional Arrays

- Consider:

```
for (i = 0; i < 100; i=i+1)
    for (j = 0; j < 100; j=j+1) {
        A[i][j] = 0.0;
        for (k = 0; k < 100; k=k+1)
            A[i][j] = A[i][j] + B[i][k] * D[k][j];
    }
```

- Must vectorize multiplication of rows of B with columns of D

- Need to access adjacent elements of B and D
- Elements of B stored in row-major order but elements of D stored in column-major order!

Stride: Multidimensional Arrays

```
for (i = 0; i < 100; i=i+1)
    for (j = 0; j < 100; j=j+1) {
        A[i][j] = 0.0;
        for (k = 0; k < 100; k=k+1)
            A[i][j] = A[i][j] + B[i][k] * D[k][j];
    }
```

- Assuming that each entry is a double word, distance between $D[0][0]$ and $D[1][0]$ is : 800 bytes
- Once vector is loaded into the register, it acts as if it has logically adjacent elements
- Use non-unit stride for D! (B uses one unit stride)
 - Ability to access non-sequential addresses and reshape them into a dense structure!
- Use LVWS/SVWS: load/store vector with stride instruction
 - Stride placed in a general purpose register (dynamic)

Problem of Stride

```
for (i = 0; i < 100; i=i+1)
    for (j = 0; j < 100; j=j+1) {
        A[i][j] = 0.0;
        for (k = 0; k < 100; k=k+1)
            A[i][j] = A[i][j] + B[i][k] * D[k][j];
    }
```

- With non-unit stride, it is possible to request accesses from the same bank frequently
- When multiple accesses compete for the same memory bank
 - Memory bank conflict!
 - Stall one access
 - Bank conflict (stall) occurs when the same bank is hit faster than bank busy time

Problem of Stride

- Example:

- 8 memory banks, bank busy time 6 cycles, total memory latency 12 cycles (startup cost, initiation)
- What is the difference between a 64-element vector load with a stride of 1 and 32?

Scatter Gather

- Sparse matrices in vector mode is a necessity
- Sparse matrix elements stored in a compact form and accessed indirectly
- Consider a sparse vector sum on arrays A and C
for ($i = 0$; $i < n$; $i=i+1$)
 $A[K[i]] = A[K[i]] + C[M[i]]$;
 where K and M are index vectors to designate the
 nonzero elements of A and C
- Gather-scatter operations are used

Scatter Gather

for ($i = 0$; $i < n$; $i=i+1$)

$A[K[i]] = A[K[i]] + C[M[i]]$;

LVI/SVI: load/store vector indexed/gather

- Use index vector:

LV	V_k, R_k	;load K
LVI	$V_a, (R_a+V_k)$	;load $A[K[]]$
LV	V_m, R_m	;load M
LVI	$V_c, (R_c+V_m)$	;load $C[M[]]$
ADDVV.D	V_a, V_a, V_c	;add them
SVI	$(R_a+V_k), V_a$	;store $A[K[]]$

A and C must have the same number of non-zero elements (size of K and M)

Vector Summary

- Vector is alternative model for exploiting ILP
 - If code is vectorizable, then simpler hardware, energy efficient, and better real-time model than out-of-order
 - More lanes, slower clock rate!
 - Scalable if elements are independent
 - If there is dependency
 - One stall per vector instruction rather than one stall per vector element
- Programmer in charge of giving hints to the compiler!
- Design issues: number of lanes, functional units and registers, length of vector registers, exception handling, conditional operations
- Fundamental design issue is memory bandwidth
 - Especially with virtual address translation and caching

Vector Summary

```
// N is the array size
double A[N+1], B[N];
... arrays are initialized ...
for(int i = 0; i < N; i++)
    A[i] = A[i+1] + B[i];
```

Can this code be vectorized?

ADD	RC, RA, 8
LV	VC, RC
LV	VB, RB
ADDV	VA, VC, VB
SV	VA, RA

Vector Summary

```
// N is the array size
double A[N+1], B[N+1];
... arrays are initialized ...
for(int i = 1; i < N+1; i++)
    A[i] = A[i-1] + B[i];
```

Will this vectorized code work correctly?

```
ADD      RC, RA, -8   ; RC = &(A[i-1])
LV       VC, RC
LV       VB, RB
ADDV     VA, VC, VB   ; A[i] = A[i-1] + B[i]
SV       VA, RA
```

Assume that $A = \{0, 1, 2, 3, 4, 5\}$; $B = \{0, 0, 0, 0, 0, 0\}$; and VLEN is 6

Vector Summary

```
for(int i = 1; i < N+1; i++)  
    A[i] = A[i-1] + B[i];
```

ADD	RC, RA, -8 ; RC = &(A[i-1])
LV	VC, RC
LV	VB, RB
ADDV	VA, VC, VB ; A[i] = A[i-1] + B[i]
SV	VA, RA

Assume that $A = \{0, 1, 2, 3, 4, 5\}$; $B = \{0, 0, 0, 0, 0, 0\}$; and VLEN is 6

Computing $A[i]$ in iteration “i” requires using the previously computed $A[i-1]$ from iteration “i-1”, which forces a serialization (you must compute the elements one at a time, and in-order).

SIMD Extensions

- Media applications operate on data types narrower than the native word size
 - Graphics systems use 8 bits per primary color
 - Audio samples use 8-16 bits
 - 256-bit adder
 - 16 simultaneous operations on 16 bits
 - 32 simultaneous operations on 8 bits

SIMD vs. Vector

- Multimedia SIMD extensions **fix the number of operands** in the opcode
 - Vector architectures have a VLR to specify the number of operands
- Multimedia SIMD extensions: **No sophisticated addressing modes** (strided, scatter-gather)
- **No mask registers**
- These features
 - enable vector compiler to vectorize a larger set of applications
 - make it harder for compiler to generate SIMD code and make programming in SIMD assembly harder

SIMD

- Implementations:
 - Intel MMX (1996)
 - Repurpose 64-bit floating point registers
 - Eight 8-bit integer ops or four 16-bit integer ops
 - Streaming SIMD Extensions (SSE) (1999)
 - Separate 128-bit registers
 - Eight 16-bit ops, Four 32-bit ops or two 64-bit ops
 - Single precision floating point arithmetic
 - Double-precision floating point in
 - SSE2 (2001), SSE3(2004), SSE4(2007)
 - Advanced Vector Extensions (2010)
 - Four 64-bit integer/fp ops

SIMD

- Implementations:
 - Advanced Vector Extensions (2010)
 - Doubles the width to 256 bits
 - Four 64-bit integer/fp ops
 - Extendible to 512 and 1024 bits for future generations
 - Operands must be consecutive and aligned memory locations

SIMD extensions

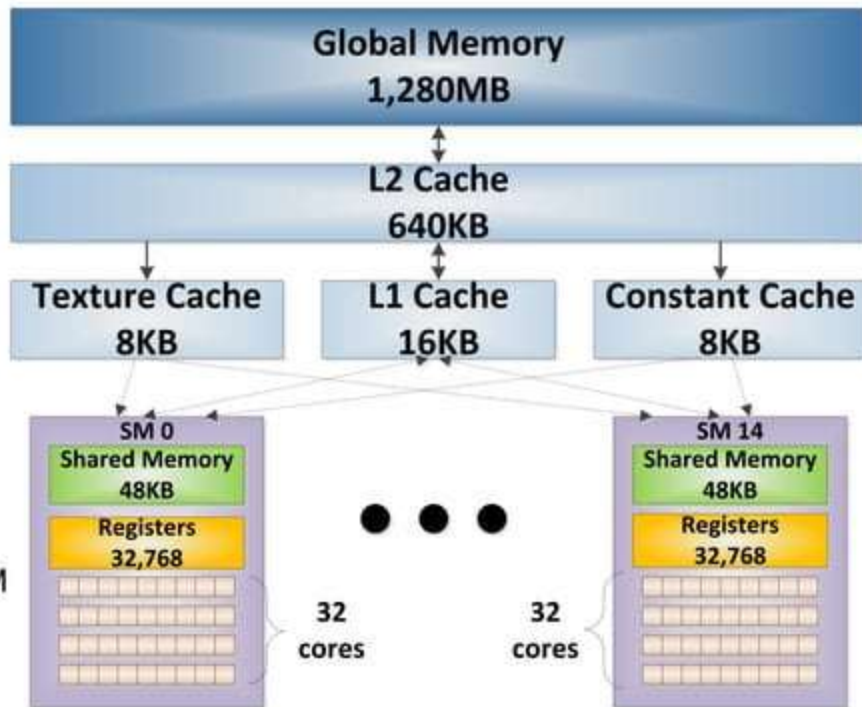
- Meant for programmers to utilize
- Not for compilers to generate
 - Recent x86 compilers
 - Capable for FP intensive apps
- Why is it popular?
 - **Costs** little to add to the standard arithmetic unit
 - **Easy** to implement
 - Need **smaller** memory bandwidth than vector
 - **Separate data transfers** aligned in memory
 - Vector: single instruction , 64 memory accesses, page fault in the middle of the vector likely!
 - Use much **smaller register** space
 - **Fewer** operands
 - No need for sophisticated mechanisms of vector architecture

Example SIMD

•Example DXPY:

L.D	F0,a	;load scalar a
MOV	F1, F0	;copy a into F1 for SIMD MUL
MOV	F2, F0	;copy a into F2 for SIMD MUL
MOV	F3, F0	;copy a into F3 for SIMD MUL
DADDIU	R4,Rx,#512	;last address to load
Loop:		
L.4D	F4,0[Rx]	;load X[i], X[i+1], X[i+2], X[i+3]
MUL.4D	F4,F4,F0	;a×X[i],a×X[i+1],a×X[i+2],a×X[i+3]
L.4D	F8,0[Ry]	;load Y[i], Y[i+1], Y[i+2], Y[i+3]
ADD.4D	F8,F8,F4	;a×X[i]+Y[i], ..., a×X[i+3]+Y[i+3]
S.4D	0[Ry],F8	;store into Y[i], Y[i+1], Y[i+2], Y[i+3]
DADDIU	Rx,Rx,#32	;increment index to X
DADDIU	Ry,Ry,#32	;increment index to Y
DSUBU	R20,R4,Rx	;compute bound
BNEZ	R20,Loop	;check if done

GTX570 GPU



Up to 1536
Threads/SM

Vector Processors vs. GPU

- Multiple functional units as opposed to deeply pipelined fewer functional units of Vector processor!
- Two level scheduling:
 - thread block scheduler and thread scheduler
- GPU (32-wide thread of SIMD instructions, 16 lanes) = Vector (16 lanes with vector length of 32) = 2 chimes

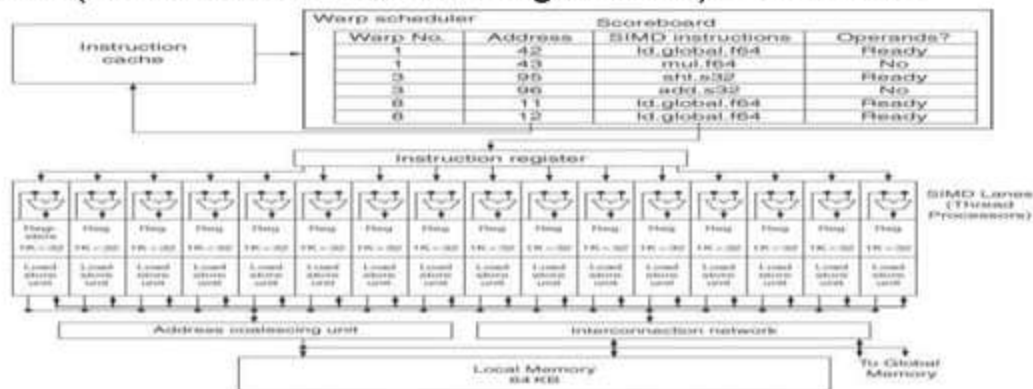
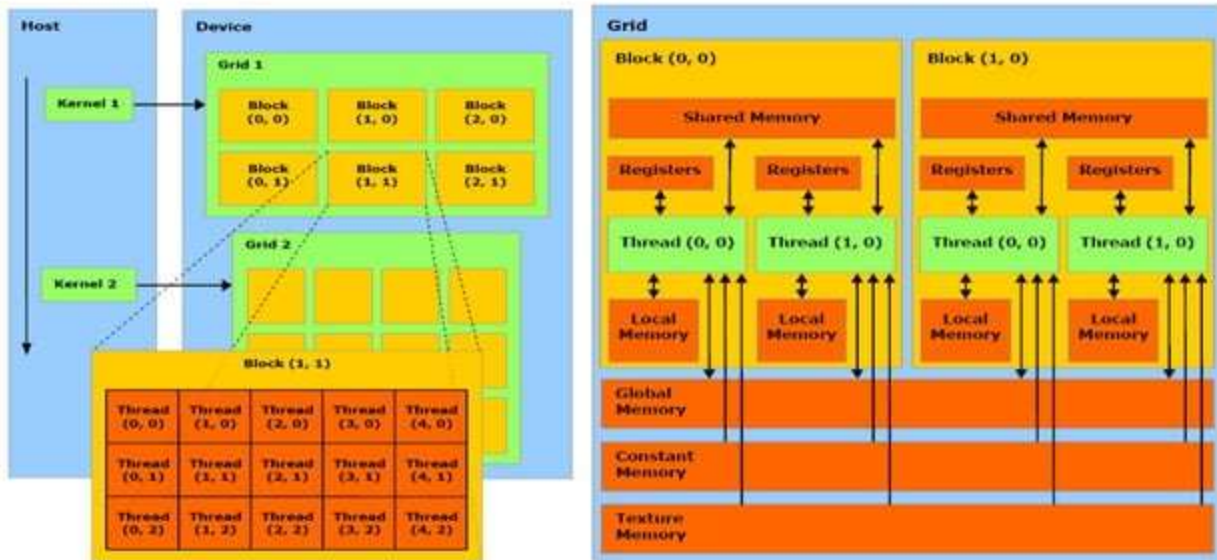


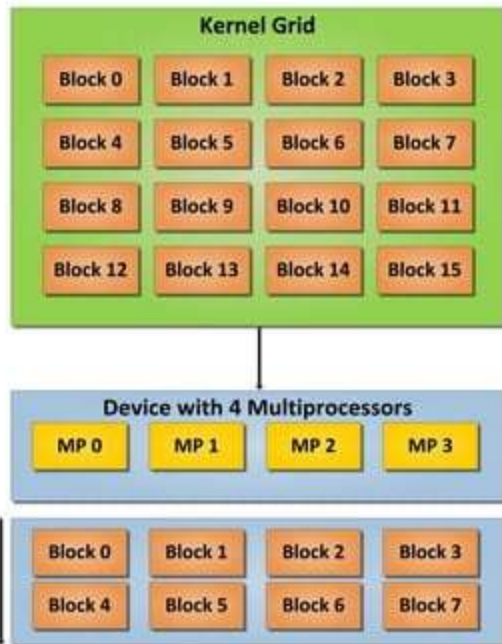
Figure 4.14 Simplified block diagram of a Multithreaded SIMD Processor. It has 16 SIMD lanes. The SIMD Thread Scheduler has, say, 48 independent threads of SIMD instructions that it schedules with a table of 48 PCs.

GTX570 GPU



- 32 threads within a block work collectively
 - ✓ Memory access optimization, latency hiding

GTX570 GPU



- Up to 1024 Threads/Block and 8 Active Blocks per SM

Vector Processors vs. GPU

- Grid and Thread Block are abstractions for programmer
- SIMD Instruction on GPU = Vector instruction on Vector
- SIMD instructions of each thread is 32 element wide
 - thread block with 32 threads =
 - Strip-minded vector loop with a vector length of 32
- Each SIMD-thread is limited to no more than 64 registers
 - 64 vector registers , each with 32-bit 32 elements
 - or 32 vector registers, each with 64-bit 32 elements
 - 32,768 threads for 16 SIMD-Lanes (2048/lane)

Vector Processors vs. GPU

- Loops:
 - Both rely on independent loop iterations
- GPU:
 - Each iteration becomes a thread on the GPU
 - Programmer specifies parallelism
 - grid dimensions and threads/block
 - Hardware handles parallel execution and thread management
 - Trick: have 32 threads/block, create many more threads per SIMD multi-processor to hide memory latency

Vector Processors vs. GPU

- Conditional Statements
 - Vector:
 - mask register part of the architecture
 - Rely on compiler to manipulate mask register
 - GPU:
 - Use hardware to manipulate internal mask registers
 - Mask register not visible to software
 - Both spend time to execute masking
- Gather-Scatter
 - GPU:
 - all loads are gathers and stores are scatters
 - Programmer should make sure that all addresses in a gather or scatter are adjacent locations

Vector Processors vs. GPU

- multithreading
 - GPU: yes
 - Vector: no
- Lanes
 - GPU: 16-32
 - A SIMD thread of 32 element wide: 1-2 chime
 - Vector 2-8
 - Vector length of 32: chime to 4 - 16
- Registers:
 - GPU (Each SIMD thread): 64 registers with 32 elements
 - Vector: 8 vector registers with 64 elements
- Latency
 - Vector: deeply pipelined, once per vector load/store
 - GPU: hides latency with multithreading

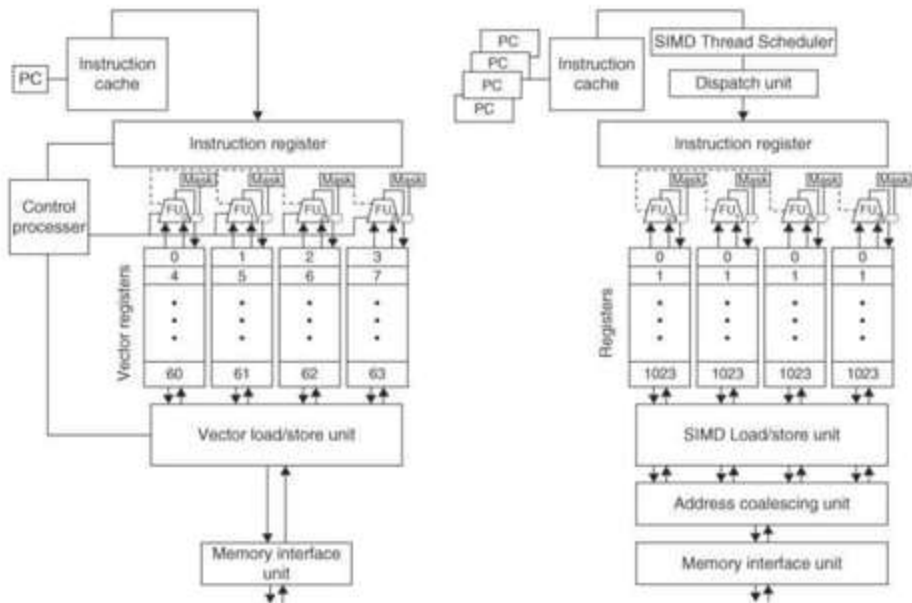


Figure 4.22 A vector processor with four lanes on the left and a multithreaded SIMD Processor of a GPU with four SIMD Lanes on the right. (GPUs typically have 8 to 16 SIMD Lanes.) The control processor supplies scalar operands for scalar-vector operations, increments addressing for unit and non-unit stride accesses to memory, and performs other accounting-type operations. Peak memory performance only occurs in a GPU when the Address Coalescing unit can discover localized addressing. Similarly, peak computational performance occurs when all internal mask bits are set identically. Note that the SIMD Processor has one PC per SIMD thread to help with multithreading.